Weekly Progress Report: GrantLens

Course Code : CSE299 | Sec : 3

Project Group No: 3

Group Members:

Anas Ibne Majbha (2322846642)
Tazreen Alam (2321506042)
Hasin Anzum Inan (2323133042)
Israt Jahan Susmi (2112280642)

Week 2: Docker Environment, Relational Schema & Dual PDF Ingestion

Sprint Goal: Transition platform architecture from Week 1 scaffolding into a containerized, asynchronous compliance evaluation engine. Establish Docker Compose with PostgreSQL 16, engineer SQLAlchemy 2.0 async models, build dual-document ingestion pipeline using PyMuPDF and pdfplumber, and deploy full Researcher Portal authentication with interactive pre-flight compliance audits.

Planned Objectives

Member	Role	Weekly Objectives
Hasin Anzum Inan ID: 2323133042	PDF Ingestion & Extraction	PyMuPDF geometry extraction (sub-millimeter margin bounds, font size distributions across spans, page counts), pdfplumber tabular financial line-item parsing, and dual-document extraction workspace UI.
Anas Ibne Majbha ID: 2322846642	Backend Core & API	FastAPI REST server orchestration, Uvicorn execution, CORS middleware configuration, heuristic compliance scoring logic, API routing (/submissions/upload), and telemetry endpoints (/api/v1/status).
Tazreen Alam ID: 2321506042	Researcher Portal (FE & BE)	Full-stack Researcher Portal: AuthPortal.jsx (tabbed auth, SSO), SubmissionPortal.jsx (audit dashboard, history table), native Bcrypt hashing, RFC 6749 JWT access/refresh tokens, and /auth/me session probe.
Israt Jahan Susmi ID: 2112280642	Docker & PostgreSQL	Multi-container Docker Compose (FastAPI + PostgreSQL 16), Debian Dockerfile with native C/C++ compilation headers (libmupdf-dev, build-essential), PostgreSQL 16 connection pooling, health checks, and volumes.

Progress Summary

Work Completed:

- Multi-container **Docker Compose** setup established with FastAPI web backend, PostgreSQL 16 database, health checks, and persistent NVMe volume.
- Production **SQLAlchemy 2.0 async models** implemented: User, GrantSubmission, Document, AuditReport, AuditFinding, and BudgetLineItem with UUID keys.
- Asynchronous **Document Ingestion Pipeline** engineered: PyMuPDF extracts geometry, font distributions, and margin bounds; pdfplumber extracts tables.
- Complete **Researcher Authentication** deployed: Bcrypt password hashing, JWT Bearer tokens (access/refresh), and /auth/me auto-validation dependency.
- **Compliance Evaluation Engine** implemented with deterministic rules: page constraints, font size thresholds, margin requirements, and section checks.
- Color-coded risk classification verified: **FATAL_ERROR (Red)**, **WARNING (Yellow)**, and **PASSED (Green)** with contextual remediation advice.
- Interactive **Researcher Portal UI** deployed: tabbed auth (demo: *researcher@grantlens.dev* / *Password123!*), dual PDF upload, and score meters.

Challenges Encountered:

- Native OS toolchain requirements: PyMuPDF and pdfplumber required specialized C/C++ compilation headers (libmupdf-dev, build-essential) in Docker.
- Passlib / Bcrypt version incompatibility: Modern bcrypt 5.0+ caused an attribute error in passlib's legacy version-checking routine.
- IPv4 vs. IPv6 loopback mismatch: Modern browsers (Brave/Chrome) resolved localhost to [::1]:8000 while server was bound to 127.0.0.1, causing fetch drops.
- Stale session tokens: Re-initialized databases caused 401 Unauthorized errors for browsers holding stale cached tokens in localStorage.

Proposed Solutions:

- Configured a multi-stage Dockerfile explicitly installing libmupdf-dev, zlib1g-dev, and build-essential packages before compiling Python wheels.
- Implemented native direct bcrypt hashing (bcrypt.hashpw / bcrypt.checkpw) with standard salt generation, eliminating passlib wrapper deprecations.
- Bound Uvicorn server to 0.0.0.0:8000 and centralized frontend API requests through a unified IPv4 loopback constant (http://127.0.0.1:8000).
- Added GET /api/v1/auth/me session probe and an automatic session purge in App.jsx plus a one-click "Sign Out & Re-Login" recovery button in SubmissionPortal.jsx.

Overall Status:

- All Week 2 milestone objectives achieved with 100% test coverage. Dual-document ingestion, database persistence, and compliance scoring validated end-to-end.

Detailed Weekly Log

Member 1: Hasin Anzum Inan led document ingestion and extraction — developed asynchronous PyMuPDF extraction for sub-millimeter margins and font distributions, implemented pdfplumber table parsing for financial rows, and designed the dual PDF upload extraction workspace.

Member 2: Anas Ibne Majbha engineered backend core runtime and API routing — configured FastAPI and Uvicorn server processes, implemented CORS middleware and error responses, authored deterministic heuristic compliance scoring logic, and managed platform telemetry.

Member 3: Tazreen Alam architected the complete Researcher Portal across frontend and backend — created AuthPortal.jsx, SubmissionPortal.jsx, and the new audit history table, and built backend authentication with native Bcrypt hashing and RFC 6749 JWT token lifecycles.

Member 4: Israt Jahan Susmi managed containerization and database infrastructure — authored the multi-stage Dockerfile with native C/C++ headers, configured docker-compose.yml for PostgreSQL 16 and FastAPI, established health checks, persistent volumes, and database connection pooling.

Report Date: October 2026

Supervisor Signature: _____